Ускорение исследования свежих данных с помощью гибких ETL-пайплайнов

Maxwell Norfolk, Dong Xie
Pennsylvania State University
mnorfolk@psu.edu · dongx@psu.edu

arXiv:2603.22220v1 [cs.DB] 23 марта 2026 г.

Аннотация

В последнее время всё острее ощущается потребность в **исследовании свежих данных** (fresh data exploration) — когда аналитики стремятся изучить основные характеристики данных или обнаружить аномалии в данных, которые активно собираются в режиме реального времени. Помимо стандартных сложностей классического исследования данных — таких как отсутствие предварительных знаний о данных или цели анализа — исследование свежих данных предъявляет особые требования к системе приёма: она должна обладать достаточной пропускной способностью, чтобы не отставать от стремительно накапливаемых данных.

Использование традиционных ETL-пайплайнов для достижения низкой задержки запросов по-прежнему остаётся чрезвычайно ресурсоёмким: им приходится выполнять избыточный объём подпрограмм предварительной обработки данных (DPR — Data Preprocessing Routines), например парсинг и индексирование, чтобы покрыть непредсказуемые характеристики данных и цели анализа.

Для преодоления этой проблемы мы предлагаем принципиально иной подход: использование временно простаивающих ресурсов или дешёвых вытесняемых ресурсов (например, Amazon Spot Instance) в процессе приёма данных. В частности, мы представляем новый тип системы приёма данных — **гибкие ETL-пайплайны** (fluid ETL pipelines), — которые позволяют пользователям запускать и останавливать произвольные DPR по требованию, не блокируя при этом приём данных.

С помощью гибких ETL-пайплайнов пользователи могут запускать потенциально полезные DPR для ускорения будущих аналитических запросов всякий раз, когда доступны простаивающие или дешёвые ресурсы. Кроме того, пользователи могут динамически переключаться между DPR при ограниченных ресурсах, адаптируясь к меняющимся интересам в ходе анализа.

Проведённые эксперименты на реальных данных подтвердили жизнеспособность нашей концепции. Введение гибких ETL-пайплайнов ставит новые задачи в области обработки произвольных запросов, генерации DPR и управления ими. В данной работе мы подробно рассматриваем открытые исследовательские проблемы и намечаем возможные пути их решения.

1. Введение

В последнее время всё больший интерес вызывает **исследование свежих данных**, при котором аналитики стремятся извлекать актуальные выводы из активно собираемых данных. В таких приложениях пользователи, как правило, практически не знают ни структуры входящих данных, ни целей будущего анализа. Поэтому им приходится формулировать запросы на ходу и уточнять их в ходе последующих итераций.

Например, во время предполагаемой кибератаки администраторы облачной инфраструктуры пытаются выяснить причину резкого роста времени отклика приложения, анализируя журналы дата-центра в реальном времени. Они могут начать с изучения распределения задержек связанных сервисов и среднего потребления CPU за последний час. Исходя из полученных результатов, они могут углубиться в другие метрики (например, категория сервиса, потребление памяти) за более ранние периоды или детализировать статистику на более мелком уровне.

В отличие от классического исследования данных, работающего со статическими или редко обновляемыми наборами данных, исследование свежих данных оперирует активными потоками для обнаружения паттернов и понимания событий по мере их возникновения в реальном времени. Для обеспечения интерактивного взаимодействия и своевременного получения актуальной информации системы исследования свежих данных сталкиваются с уникальными вызовами: им необходимо одновременно обеспечивать достаточную пропускную способность для приёма данных и поддерживать низкую задержку запросов.

К сожалению, обеспечить идеальный опыт исследования свежих данных по-прежнему чрезвычайно дорого. С одной стороны, можно сохранять входящие данные различных форматов (JSON, CSV, текст) без какой-либо предобработки и обрабатывать произвольные запросы непосредственно по сырым данным. Однако, несмотря на максимально возможную пропускную способность при приёме, аналитические запросы страдают от высокой задержки: значительные усилия по подготовке данных откладываются до момента выполнения запроса.

С другой стороны, можно применять ETL-пайплайны, которые выполняют заранее определённый набор **подпрограмм предварительной обработки данных (DPR)** — таких как парсинг и построение индексов — в процессе приёма. Оплачивая эти накладные расходы заранее, будущие запросы могут использовать структуры, построенные DPR (разобранные поля, индексы, материализованные представления и т. д.), что эффективно снижает задержку. Однако каждая DPR требует выделенных вычислительных ресурсов, чтобы скорость приёма данных соответствовала скорости их сбора. Помимо этого, непредсказуемый характер исследования свежих данных может привести к несоответствию между заранее определёнными DPR и последующими запросами.

Распространённый способ обойти эту проблему — исчерпывающе выполнять все потенциально полезные DPR в ETL-пайплайнах, покрывая все возможные запросы. Тем не менее такой подход приводит к значительным и излишним накладным расходам по ресурсам.

Для преодоления этой проблемы мы предлагаем иной подход: использование временно простаивающих ресурсов или дешёвых вытесняемых ресурсов для адаптивного выполнения DPR, способных ускорить будущие запросы. По мере изменения скорости и характеристик сбора данных системы приёма должны обеспечивать достаточно ресурсов для пиковой нагрузки. В результате в непиковые периоды в системе может накапливаться значительный объём простаивающих ресурсов. Кроме того, современные облачные провайдеры предлагают неиспользуемые мощности (например, EC2 Spot Instances) с существенной скидкой (до 90%), допуская оппортунистическое развёртывание сервисов при низких затратах.

Ещё один ключевой принцип, которым мы руководствуемся: пользователи, как правило, интересуются лишь небольшим, но меняющимся подмножеством данных. Поэтому минимальные усилия DPR, необходимые для достижения хорошей производительности запросов, значительно меньше суммарных усилий всех потенциально полезных DPR. Исходя из этого, мы видим возможность ускорить исследование свежих данных за счёт целенаправленного адаптивного инвестирования ресурсов в значимые операции (DPR), которые в конечном счёте ускорят будущие запросы.

---

2. Представляем гибкие ETL-пайплайны

В данном разделе мы даём определение гибкого ETL-пайплайна и описываем конкретный план его реализации. Затем показываем, как гибкие ETL-пайплайны могут ускорить рабочие нагрузки по исследованию свежих данных за счёт простаивающих ресурсов — с помощью эксперимента на реальных данных. После этого мы описываем ключевые компоненты, которые необходимо создать вокруг гибкого ETL-пайплайна для построения практичного решения. Наконец, мы рассматриваем связь и различия между существующими решениями и нашим подходом.

Определение гибких ETL-пайплайнов

Подобно традиционным ETL-пайплайнам, цель гибкого ETL-пайплайна состоит в том, чтобы принимать входящий поток данных, избирательно извлекать необходимую информацию из сырых данных, преобразовывать её в другой формат, строить и поддерживать вспомогательные структуры и/или загружать их в базу данных.

В общем случае мы определяем **подпрограмму предварительной обработки данных (DPR)** как процедуру, которая принимает набор входящих данных и создаёт (или поддерживает) некоторые структуры (индексы, материализованные представления и т. д.) для ускорения будущих запросов. Например, DPR может принимать отдельную JSON-запись, разбирать из неё несколько полей, вычислять функцию от извлечённых значений и вставлять запись в индекс. Другой пример DPR — принять набор CSV-записей, извлечь все поля каждой записи и переформатировать их в Parquet.

В таком понимании традиционный ETL-пайплайн можно рассматривать как систему приёма, которая непрерывно применяет заранее определённый набор DPR к входящим записям. В

отличие от него, DPR, выполняемые в гибком ETL-пайплайне, **не определяются заранее** и могут меняться в зависимости от доступности ресурсов и потребностей пользователей.

Конкретно, гибкий ETL-пайплайн должен обеспечивать как минимум следующие возможности:

1. **Запуск и остановка произвольных DPR в любой момент без блокировки приёма.** Для этого гибкий ETL-пайплайн применяет DPR только к тем записям, которые поступили в период её активности.
2. **Сохранение информации об активных периодах** всех предыдущих и выполняемых в настоящее время DPR, а также о расположении построенных ими структур.
3. **Сохранение сырой копии принятых данных** для корректной обработки любых произвольных запросов.

Как только данные принимаются гибким ETL-пайплайном, их сырая форма и все построенные для них структуры должны сразу становиться доступными для обработчика запросов, чтобы обеспечить актуальность результатов.

Поскольку DPR применяется только к данным, принятым в период её активности, структуры, которые она строит или поддерживает, могут охватывать лишь часть принятых данных. Это означает, что такие структуры могут быть **частичными** по отношению к произвольному запросу и не покрывать все связанные с ним данные. Как мы обсудим в разделе 4, частичные структуры могут тем не менее обеспечить значительный выигрыш в производительности при использовании новых методов планирования и оптимизации запросов.

Реализация гибкого ETL-пайплайна

Для реализации работающего гибкого ETL-пайплайна достаточно объединить систему потоковой передачи событий (например, Kafka) с несколькими традиционными ETL-пайплайнами (реализованными через Spark, Flink и т. д.). Между системой потоковой передачи и ETL-пайплайнами вводится промежуточный слой (middleware), который управляет маршрутизацией данных.

Когда пользователь хочет добавить новую DPR, он реализует соответствующий традиционный ETL-пайплайн, после чего middleware инициирует его и просит систему потоковой передачи реплицировать и направлять туда данные, поступившие с этого момента. При отключении DPR middleware прекращает отправку данных в соответствующий пайплайн и останавливает его. Middleware также отвечает за отслеживание соответствия между DPR, построенными ими структурами и временными периодами их работы.

Выигрыш в производительности от гибких ETL-пайплайнов

Для демонстрации выигрыша в производительности мы провели эксперимент на реальном потоке: принимали данные журнала событий GitHub за сутки (события записаны в формате JSON) и симулировали простой сценарий исследования свежих данных.

> **Пример 2.1.** Алиса пытается выяснить причину текущей проблемы с производительностью при создании pull request на GitHub, анализируя активный поток

> событий. Сначала она просматривает pull request по всем репозиториям и выявляет репозиторий с аномально высоким числом недавних PR. Затем она изучает все последние события из этого репозитория и обнаруживает, что конкретный аккаунт спамит новыми событиями. После этого она проверяет другие репозитории, в которых активен этот аккаунт, чтобы определить, является ли он вредоносным.

Мы выполняли следующие запросы к потоку данных в процессе его приёма:

- **(Q1)** найти три репозитория с наибольшим числом pull request;
- **(Q2)** найти трёх наиболее активных пользователей для конкретного репозитория;
- **(Q3)** найти три репозитория, в которых данный пользователь наиболее активен.

Мы сравнивали три стратегии ETL в сочетании с оптимизированными вручную запросами:

- **Baseline ETL:** никаких дополнительных DPR — только базовый парсинг стандартных полей. Лишь Q1 выигрывает от построенного индекса; Q2 и Q3 вынуждены обращаться к сырым данным.
- **Fluid ETL:** гибкий ETL-пайплайн, который запускает дополнительные DPR при наличии простаивающих ресурсов CPU и адаптирует набор выполняемых DPR по мере изменения запросов. Например, с 1-го по 8-й час строятся хэш-индексы по языку программирования и типу pull request; при переходе к Q2 — предфильтрация по идентификатору репозитория; при Q3 — замена предфильтра на фильтр по идентификатору аккаунта.
- **Extensive ETL:** к базовому пайплайну добавляются шесть новых DPR, охватывающих все поля, необходимые для Q1–Q3, на протяжении всего периода приёма — симуляция сценария «покрыть все мыслимые запросы».

Результаты эксперимента наглядно показывают, что ни Baseline ETL, ни Extensive ETL не являются оптимальными на практике. Q2 и Q3 страдают от существенно более высокой задержки при использовании только базового пайплайна. Extensive ETL требует значительно больше ресурсов CPU (до 2x), чем выделено. Гибкий ETL, напротив, обеспечивает лучшее из обоих подходов: система никогда не превышает выделенный лимит CPU в процессе приёма и при этом доставляет значительно более высокую производительность запросов (в большинстве случаев >3x по сравнению с Baseline ETL).

На пути к практичному исследованию свежих данных

Для разработки полноценного практичного решения необходимо решить следующие задачи:

1. Прогнозировать будущие нагрузки и генерировать эффективные DPR, строящие структуры для их ускорения.
2. Тщательно планировать произвольные запросы для максимального использования частичных структур.
3. Принимать динамические решения о запуске и остановке DPR для баланса между пропускной способностью приёма и задержкой запросов при ограниченных ресурсах.

В описанном эксперименте все эти задачи выполнялись вручную. Поскольку большинство аналитиков данных не являются системными специалистами, крайне важно разработать

автоматизированные инструменты для их решения.

Сравнение с существующими решениями

Многочисленные методы обработки запросов непосредственно по сырым данным широко применяются в современных озёрах данных. Эти методы откладывают объёмную подготовку данных до момента выполнения запроса, что приводит к высокой задержке. С гибкими ETL-пайплайнами большинство необходимых подготовительных операций можно предсказать и выполнить заранее, однако обработка по сырым данным по-прежнему нужна, когда ранее созданные DPR не покрывают требуемые структуры.

Такие методы, как database cracking и speculative loading, постепенно извлекают информацию и строят структуры во время выполнения запросов. Они рассчитаны на схожие паттерны запросов в будущем. Однако при столкновении с принципиально иным типом запроса они не дают никакого выигрыша. Наш подход, напротив, проактивно предсказывает будущие запросы и строит структуры заранее.

Системы стриминга могут отслеживать результаты постоянных запросов над предопределёнными (скользящими или опрокидывающимися) окнами. Тем не менее в исследовательском контексте пользователям может понадобиться выполнять запросы с различными временными окнами или даже по историческим данным, где стриминговые системы бессильны.

---

3. Генерация эффективных DPR

Прогнозирование будущих нагрузок и автоматическая генерация эффективных DPR для их ускорения — ключевая задача. Подобно классическим задачам исследования данных, пользователи, вероятно, будут формулировать запросы со структурами и предикатами, похожими на предыдущие. Таким образом, мы можем начать с предсказания будущих нагрузок путём пропорционального смешивания шаблонов запросов, встречавшихся в недавнем прошлом.

С учётом последних достижений в области рассуждательных способностей **больших языковых моделей (LLM)**мы видим возможность разработать продвинутые модели предсказания, способные прогнозировать наиболее вероятные следующие шаги в процессе исследования. При этом важную роль могут играть результаты предыдущих запросов: разные наблюдения могут приводить к противоположным аналитическим решениям. Таким образом, помимо итоговой цели и истории запросов, модель рассуждений должна принимать во внимание и результаты запросов.

После получения набора потенциальных запросов для ускорения необходимо проанализировать их логические планы и выявить структуры, которые могут им помочь. Поскольку несколько запросов могут выигрывать от одних и тех же структур, следует отдавать приоритет тем, которые помогают наибольшему числу потенциальных запросов. Для идентификации таких приоритетных структур можно применять классические методы выбора материализованных представлений и индексов.

Совместное использование вычислений несколькими DPR

При одновременном выполнении нескольких DPR между ними может быть значительное пересечение логики. Например, две DPR, работающие с комментариями к задачам в потоке событий GitHub, обе должны фильтровать события создания задач и разбирать поле `comments` из сложной JSON-структуры. Если эти DPR выполняются независимо, одна и та же логика будет применяться к потоку данных дважды, приводя к излишним накладным расходам.

Эксперимент с тремя уровнями оптимизации показал:

- **Уровень 0** (нет оптимизаций): 36 КБ/с.
- **Уровень 1** (общий парсинг JSON и фильтрация): 38 КБ/с (+5%).
- **Уровень 2** (дополнительное совместное использование логики NLP — токенизации, распознавания именованных сущностей): **78 КБ/с** — пропускная способность более чем удвоилась.

Для объединения нескольких DPR в одну с устранением избыточных вычислений можно использовать новое высокоуровневое промежуточное представление через фреймворк **MLIR** с кастомными правилами оптимизации.

---

4. Планирование запросов с частичными структурами

Структуры, построенные гибкими ETL-пайплайнами, часто являются **частичными** по отношению к произвольным запросам — ввиду ограниченности ресурсов и адаптивного управления DPR. В таких случаях классические методы оптимизации запросов неприменимы, поскольку требуют, чтобы используемые структуры покрывали все данные, связанные с запросом.

Важно подчеркнуть, что концепция частичных структур в данной работе принципиально отличается от понятия частичного индекса в классической литературе. Классический частичный индекс строится только на записях, удовлетворяющих заранее заданному условию, и потому является частичным по отношению ко всей таблице. Частичные структуры в нашем смысле являются частичными по отношению к данным, необходимым для ответа на конкретный запрос.

Сшивание планов (Plan Stitching)

Ключевая идея решения этой проблемы — **декомпозиция данных на несколько разделов**, переписывание исходного запроса в виде подзапросов для различных разделов и объединение их результатов. После декомпозиции классические методы оптимизации могут использовать частичные структуры для ускорения части подзапросов.

Эксперимент с запросом Q2 и частичным хэш-индексом по `repo.id` наглядно продемонстрировал пользу сшивания планов. Q2 выполнялся над событиями за различные временные окна (2, 4, 6 часов и т. д.), тогда как индекс охватывал только последние 10 часов приёма данных. Мы декомпозировали Q2 на две части: одна работала с данными за

последние 10 часов с использованием хэш-индекса, другая — непосредственно по оставшимся сырым данным. Слияние результатов давало значительное ускорение по сравнению с обработкой только по сырым данным.

На практике нас ждут значительно более сложные сценарии: для данного произвольного запроса может существовать несколько частичных структур для его ускорения, а их покрытие может быть произвольным и перекрывающимся. Поиск наилучшей стратегии сшивания планов для полного использования доступных частичных структур остаётся **открытой исследовательской проблемой**.

---

5. Адаптивное управление DPR

На практике многие приложения работают при ограниченных ресурсах или бюджетных ограничениях. Каждая запускаемая DPR в гибком ETL-пайплайне представляет собой инвестицию ресурсов в процессе приёма данных с расчётом на то, что построенные ею структуры ускорят будущие запросы. При ограниченных простаивающих или дешёвых ресурсах нельзя иметь слишком много параллельных инвестиций, иначе система приёма может не справляться с потоком данных. Поэтому при изменении доступности ресурсов и прогнозируемой нагрузки необходимо принимать динамические решения.

Для количественной оценки стоимости и пользы DPR необходимо выстроить модели:

- **Оценка стоимости:** поскольку разные типы ресурсов (CPU, дисковая пропускная способность) могут быть узким местом, требуются оценки по каждому ресурсу. Для работающей DPR можно собирать базовую статистику выполнения; для новой — запускать её на небольшой выборке потока данных.
- **Оценка пользы:** для оценки выигрыша, который DPR принесёт произвольным запросам, можно использовать модели стоимости из оптимизации запросов.

Имея эти оценки, можно провести **оптимизацию выбора DPR** для заданной рабочей нагрузки запросов при текущих ресурсных ограничениях. Описанная оптимизация является экземпляром **задачи о рюкзаке (knapsack problem)**, известной как NP-полной. Поэтому для получения хорошего решения в разумные сроки необходимо разрабатывать более эффективные приближённые алгоритмы.

Помимо периодической переоптимизации, метод, основанный на **теории управления** (control theory), может обеспечить более инкрементальный подход: по мере изменения нагрузки новые DPR вводятся для новых типов запросов, тогда как те, что не обслуживают ни один тип запросов из прогнозируемой нагрузки, отключаются.

---

6. Заключение

В данной работе мы указываем на избыточную стоимость существующих решений для идеального исследования свежих данных. Опираясь на ключевое наблюдение о

возможности использования простаивающих ресурсов системы и/или дешёвых вытесняемых ресурсов, мы предлагаем принципиально иной подход: спекулятивный запуск DPR для ускорения рабочих нагрузок исследования свежих данных.

В частности, мы представляем новый тип системы приёма данных — **гибкие ETL-пайплайны** — способные запускать и останавливать произвольные DPR по требованию, не блокируя приём данных. Жизнеспособность нашей концепции подтверждена экспериментом над реальными данными. На пути к исчерпывающему и практичному решению мы выявили несколько ключевых проблем в области генерации DPR, обработки запросов и адаптивного управления DPR, детально обсудили их и наметили возможные направления решения.

В дальнейшей работе мы планируем разработать базовые методологии для решения этих новых задач и создать комплексную практичную систему для облегчения исследования свежих данных.

Список литературы

1. GH Archive. https://www.gharchive.org/ (2024).
2. Amazon EC2 Spot Instance. https://aws.amazon.com/ec2/spot/ (2025).
3. Apache Flink. https://flink.apache.org/ (2025).
4. Apache Kafka. https://kafka.apache.org (2025).
5. Apache Parquet. https://parquet.apache.org/ (2025).
6. Apache Spark. https://spark.apache.org/ (2025).
7. Data Build Tool (dbt). https://www.getdbt.com (2025).
8. LLVM Just-in-Time (JIT) Compiler. https://llvm.org/docs/ORCv2.html (2025).
9. Multi-Level Intermediate Representation (MLIR). https://mlir.llvm.org (2025).
10. Agiwal A. et al. *Napa: Powering Scalable Data Warehousing with Robust Query Performance at Google.* PVLDB 14(12), 2021.
11. Alagiannis I. et al. *NoDB: efficient query execution on raw data files.* ACM SIGMOD, 2012.
12. Armbrust M. et al. *Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark.* ACM SIGMOD, 2018.
13. Assi M., Haraty R. *A Survey of the Knapsack Problem.* ACIT, 2018.
14. Cheng Y., Rusu F. *Parallel in-situ data processing with speculative loading.* SIGMOD, 2014.
15. Cocke J. *Global common subexpression elimination.* Symposium on Compiler Optimization, 1970.
16. Fang Y. et al. *UDP: a programmable accelerator for extract-transform-load workloads and more.* MICRO, 2017.
17. Ge C. et al. *Speculative Distributed CSV Data Parsing for Big Data Analytics.* ACM SIGMOD, 2019.
18. Halim F. et al. *Stochastic Database Cracking.* PVLDB 5(6), 2012.
19. Idreos S., Kersten M.L., Manegold S. *Database Cracking.* CIDR, 2007.
20. Karpathiotakis M. et al. *Adaptive Query Processing on RAW Data.* PVLDB 7(12), 2014.

21. Khelifati A. et al. *TSM-Bench: Benchmarking Time Series Database Systems for Monitoring Applications.* PVLDB 16(11), 2023.
22. Li Y. et al. *Mison: A Fast JSON Parser for Data Analytics.* PVLDB 10(10), 2017.
23. Mühlbauer T. et al. *Instant Loading for Main Memory Databases.* PVLDB 6(14), 2013.
24. Palkar S. et al. *Filter Before You Parse: Faster Analytics on Raw Data with Sparser.* PVLDB 11(11), 2018.
25. Seshadri P., Swami A.N. *Generalized Partial Indexes.* ICDE, 1995.
26. Shukla D. et al. *Schema-Agnostic Indexing with Azure DocumentDB.* PVLDB 8(12), 2015.
27. Solleza F. et al. _Mach: Firefighting Time-Critical Issues in Complex Systems Using High-Frequency Telemetry._PVLDB 17(12), 2024.
28. Stonebraker M. *The Case for Partial Indexes.* SIGMOD Rec. 18(4), 1989.
29. Xie D. et al. *FishStore: Faster Ingestion with Subset Hashing.* ACM SIGMOD, 2019.